

### Experiment no. 03

**Title:** Implementation of Midpoint Circle Drawing Algorithm

#### Objective:

The primary objective of this experiment is to implement the Midpoint Circle Drawing Algorithm. This algorithm is used for efficiently drawing circles by determining the next pixel based on the previous one using integer calculations, minimizing the use of floating-point arithmetic. The algorithm will demonstrate the fundamentals of drawing circles in a raster scan display.

#### Algorithm:

- *Input:* Center of the circle (  $(x_c, y_c)$  ) and radius (  $r$  ).
- *Initialize:* Set the starting point of the circle at (  $(x, y) = (0, r)$  ), and calculate the initial decision parameter as (  $p_0 = 1 - r$  ).
- *Loop:* Continue iterating while (  $x \leq y$  ):
  - Plot the current point and its reflections in all 8 octants of the circle.
  - Update the decision parameter (  $p_k$  ):
    - If (  $p_k < 0$  ), update (  $p_{k+1} = p_k + 2x + 1$  ).
    - Else, update (  $p_{k+1} = p_k + 2x - 2y + 1$  ), and decrement (  $y$  ).
  - Increment (  $x$  ).
- *Output:* Display the circle on the screen.

#### Code:

```
#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;
float x_c, y_c, r;
void plotCirclePoints(float x, float y) {
    glBegin(GL_POINTS);
    glVertex2f(x_c + x, y_c + y);
    glVertex2f(x_c + x, y_c - y);
    glVertex2f(x_c - x, y_c + y);
    glVertex2f(x_c - x, y_c - y);

    glVertex2f(x_c + y, y_c + x);
    glVertex2f(x_c + y, y_c - x);
    glVertex2f(x_c - y, y_c + x);
    glVertex2f(x_c - y, y_c - x);
}
```

```

    glEnd();
}
void drawCircle() {
    float x = 0;
    float y = r;
    float pk = (5.0 / 4.0) - r;
    // +y -> +x
    while (x <= y) {
        plotCirclePoints(x, y);
        cout<<x<<"\t"<<y<<endl;
        if (pk < 0) {
            pk += 2 * x + 1;
        } else {
            y--;
            pk += 2 * x - 2 * y + 1;
        }
        x++;
    }
}
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 0.0); // Yellow color for axes
    //drawGrid(800, 600); // Example screen size
    glColor3f(1.0, 1.0, 1.0); // White color for circle
    drawCircle();
    glFlush();
}

```

```

void init() {
    glClearColor(0.0, 0.0, 0.0, 1.0); // Black
    background
    //glColor3f(1.0, 1.0, 1.0); // White color for grid
    glPointSize(2.0); // Set point size
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-400, 400, -300, 300); // Set the
    coordinate system
}

int main(int argc, char** argv) {

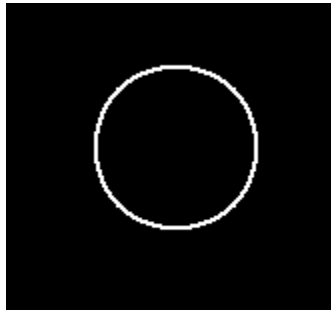
    cout << "Enter the value of center point: ";
    cin >> x_c >> y_c;
    cout << "Enter the value of radius: ";
    cin >> r;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 600); // Set window size
    glutCreateWindow("Midpoint Circle Drawing");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

**Output:**

```
Enter the value of center point: 5 19
Enter the value of radius: 40
```

**Discussion:**

The algorithm utilizes the symmetry of a circle to reduce computational overhead. By calculating points for just one octant and reflecting these in other octants, it minimizes the number of calculations. So, to plot the points for the reflected octants calculations are little bit hazy to calculate & write. The decision parameter  $p_k$  helps determine the next pixel to plot. This reduces the use of square roots and floating-point arithmetic, making the algorithm efficient. But, it causes some issues to write code. Handling floating-point precision in functions, though mostly handled by integer arithmetic here, can sometimes lead to minor visual inconsistencies. Since OpenGL uses a Cartesian coordinate system, transformations are necessary to map pixels correctly, especially for non-centered circles

**Experiment no. 04**

**Title:** 2D Geometric Transformations

**Objective:**

The objective of this lab is to understand and implement basic 2D geometric transformations on polygons. The transformations include translation, scaling, rotation, reflection, and shearing. By using these transformations, the program modifies the position, size, and orientation of a polygon and visualizes the effects interactively.

**Algorithm:**

- *Input:* The user enters the number of vertices for the polygon and the coordinates of each vertex.
- *Display:* The polygon is drawn on the screen using OpenGL.
- *Transformation Selection:* The user selects a transformation (translation, scaling, rotation, reflection, or shearing) from a menu.
- *Apply Transformation:*

- The selected transformation is applied to the vertices of the polygon using the respective mathematical formulas.
- The transformed polygon is stored in a new vector and displayed in a different color.
- *Output:* The transformed polygon is rendered on the screen alongside the original polygon for comparison.
- *Loop:* The user can repeat the process by selecting different transformations.

**Code:**

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>vector&lt;Point&gt; original; vector&lt;Point&gt; transformed;  void drawPolygon(const std::vector&lt;Point&gt;&amp; vertices) {      glBegin(GL_POLYGON);      for (const auto&amp; vertex : vertices) {         glVertex2f(vertex.x, vertex.y);     }      glEnd(); }  void display() {      glClear(GL_COLOR_BUFFER_BIT);      glColor3f(1.0, 0.0, 0.0); // Original polygon in red      drawPolygon(original);      glColor3f(0.0, 1.0, 0.0); // Transformed polygon in green      drawPolygon(transformed);      glutSwapBuffers(); }  void initialize() {</pre> | <pre>glClearColor(1.0, 1.0, 1.0, 1.0); // Set background color to white      gluOrtho2D(-100, 100, -100, 100); }  void applyTranslation(float tx, float ty) {      transformed.clear();      for (const auto&amp; pt : original) {          Point newPt;          newPt.x = pt.x + tx;          newPt.y = pt.y + ty;          transformed.push_back(newPt);      }      glutPostRedisplay(); // Update the display after transformation }  void applyScaling(float sx, float sy, float fx, float fy) {      transformed.clear();      for (const auto&amp; pt : original)      {          Point newPt;          newPt.x = (pt.x - fx) * sx + fx;</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```

newPt.y = (pt.y - fy) * sy + fy;

    transformed.push_back(newPt); }

    glutPostRedisplay(); // Update the display after
transformation}

void applyRotation(float angle, int pivotX, int
pivotY){

    transformed.clear();

    float radianAngle = (angle * 3.1416) / 180.0;

    for (const auto& p : original) {

        float translateX = p.x - pivotX;

        float translateY = p.y - pivotY;

        float rotationX = translateX * cos(radianAngle) -
translateY * sin(radianAngle);

        float rotationY = translateX * sin(radianAngle) +
translateY * cos(radianAngle);

        Point newPoint;

        newPoint.x = rotationX + pivotX;

        newPoint.y = rotationY + pivotY;

        transformed.push_back(newPoint); }

    glutPostRedisplay(); // Update the display after
transformation}

void applyReflection(char option){

    transformed.clear();

    for (const auto& p : original) {

        Point newPoint;

        if(option == 'a')    {

            newPoint.x = p.x;

            newPoint.y = -p.y;    }

    else if(option == 'b')    {

        newPoint.x = -p.x;

        newPoint.y = p.y; }

    else if(option == 'c') {

        newPoint.x = p.y;

        newPoint.y = p.x;    }

    else    {

        newPoint.x = -p.x;

        newPoint.y = -p.y;    }

        transformed.push_back(newPoint); }

    glutPostRedisplay(); // Update the display after
transformation}

void applyShearing(float sh, char option){

    transformed.clear();

    for (const auto& p : original) {

        Point newPoint;

        if(option == 'X')    {

            newPoint.x = p.x + (sh * p.y);

            newPoint.y = p.y;}

        else    {

            newPoint.x = p.x;

            newPoint.y = p.y + (sh * p.x);    }

        transformed.push_back(newPoint); }

    glutPostRedisplay(); // Update the display after
transformation}

void showMenu() {

    cout << "2D Geometric Transformation" << endl;

    cout << "....." << endl;

    cout << "1 -> Translation " << endl;

    cout << "2 -> Scaling (Fixed Point)" << endl;

    cout << "3 -> Rotation (Rotation / Pivot Point)" <<
endl;

    cout << "4 -> Reflection " << endl;

```

```

cout << "5 -> Shearing" << endl;

    cout << "6 -> Quit" << endl;

    cout << "Enter your option: ";
void getVertices() {

    cout << "Enter the number of vertices: ";

    int n;

    cin >> n;

    original.resize(n);

    cout << "Enter the coordinates of each vertex
(x,y): " << endl;

    for (int i = 0; i < n; ++i) {

        cout << i + 1 << " -> ";

        cin >> original[i].x >> original[i].y;    }

    glutPostRedisplay(); // Update the display with
the new polygon}
void processInput(int option) {

    switch (option) {

        case 1: {

            cout <<
"=====Translation=====
" << endl;

            getVertices();

            float tx, ty;

            cout << "Enter translation factors (tx,ty): ";

            cin >> tx >> ty;

            applyTranslation(tx, ty);

            break;    }

        case 2: {

            cout <<
"=====Scaling=====
" <<
endl;

```

```

getVertices();

        float sx, sy, fx, fy;

        cout << "Enter the value of Scaling factor
(SX,SY) : " << endl;

        cin >> sx >> sy;

        cout << "Enter the fixed point (fX,fY): " <<
endl;

        cin >> fx >> fx;

        applyScaling(sx, sy, fx, fy);

        break;    }

        case 3: {

            cout <<
"=====Rotation=====
" << endl;

            getVertices();

            float angle, xr, yr;

            cout << "Enter the value of the Theta (Angle)
for Rotation (Degree): " << endl;

            cin >> angle;

            cout << "Enter the Pivot/Rotation point
(xr,yr): " << endl;

            cin >> xr >> yr;

            applyRotation(angle, xr, yr);

            break;    }

        case 4: {

            cout <<
"=====Reflection=====
" << endl;

            getVertices();

            char axis;

            cout << "Enter the axis of reflection (a -> X-
axis, b -> Y-axis, c -> Origin, d -> X=Y axis): ";

```

```

cin >> axis;

    applyReflection(axis);

    break;    }

case 5: {

    cout <<
"=====Shearing=====
<< endl;

    getVertices();

    float sh;

    char axis;

    cout << "Enter the shearing factors (X for x-
axis, Y for y-axis): ";

    cin >> axis;

    cout << "Enter the Shearing value : ";

    cin >> sh;

    applyShearing(sh, axis);

    break;    }

case 6:

    cout << "Quit" << endl;

    exit(0);

default:

    cout << "Invalid option. Please try again." <<
endl;

    break;    }    }

void menuLoop() {

    while (true) {

        showMenu();

        int option;

        cin >> option;

        if (option == 6) {

            exit(0);}

```

```

processInput(option);

    }

}

int main(int argc, char** argv) {

    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_DOUBLE |
GLUT_RGB);

    glutInitWindowSize(600, 600);

    glutInitWindowPosition(100, 100);

    glutCreateWindow("Polygon Transformation");

    initialize();

    glutDisplayFunc(display);


    // Start the menu loop in a separate thread
    std::thread menuThread(menuLoop);

    menuThread.detach();


    glutMainLoop();

    return 0;

}

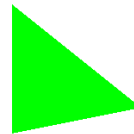
```

## Output:

```

2D Geometric Transformation
.....
1 -> Translation
2 -> Scaling (Fixed Point)
3 -> Rotation (Rotation / Pivot Point)
4 -> Reflection
5 -> Shearing
6 -> Quit
Enter your option: 4
=====Reflection=====
Enter the number of vertices: 3
Enter the coordinates of each vertex (x,y):
1 -> 20 90
2 -> 20 40
3 -> 70 80
Enter the axis of reflection (a -> X-axis, b -> Y-axis, c -> Origin, d -> X=Y axis): a
2D Geometric Transformation
.....
1 -> Translation
2 -> Scaling (Fixed Point)
3 -> Rotation (Rotation / Pivot Point)
4 -> Reflection
5 -> Shearing
6 -> Quit
Enter your option:

```



## Discussion:

Several challenges arised when implemented 2D geome tric transformations. Each transformation has its specific mathematical formulas. Some mistakes such as incorrect scaling or rotation formulas, particularly when dealing with fixed points or pivot points. And, Forgetting to clear the screen for visual, where multiple polygons overlap without proper differentiation between the original and transformed shapes. I forgot to convert degree to radian, which caused a huge problem. It was hard to manage different variables for different equations. Sometimes, I forgot the required variable name which store data. Then, I did search back. At last, to do the continuous option & displaying draw was not easy.

## Experiment no. 05

**Title:** Implementing Cohen-Sutherland Line Clipping Algorithm

### Objective:

The Cohen-Sutherland algorithm is a popular method used in computer graphics to efficiently clip a line segment against a rectangular clipping window. This implementation allows users to visualize the line segment and its clipped portion within a specified clipping area.

### Algorithm:

The Cohen-Sutherland algorithm follows these steps to achieve line clipping:

- *Input Specification:*
  - The user is prompted to enter the coordinates of two endpoints of the line segment. These coordinates are represented as  $(x_0, y_0)$  for the starting point and  $(x_1, y_1)$  for the endpoint.
- *Define the Clipping Window:*
  - The clipping window is defined with the following boundaries:
    - Left boundary at  $x_{min} = 20$
    - Right boundary at  $x_{max} = 80$

- Bottom boundary at  $y_{\min} = 20$
  - Top boundary at  $y_{\max} = 80$
- *Assign Region Codes:*
  - Each endpoint of the line segment is assigned a 4-bit region code that indicates its position relative to the clipping window. The region codes are defined as follows:
    - **INSIDE (0000):** The point is inside the clipping window.
    - **LEFT (0001):** The point is to the left of the window.
    - **RIGHT (0010):** The point is to the right of the window.
    - **BOTTOM (0100):** The point is below the window.
    - **TOP (1000):** The point is above the window.
  - The computeCode function calculates the region code for each point based on its coordinates.
- *Clipping Process:*
  - The algorithm iteratively checks the region codes of both endpoints:
    - *Both Points Inside:* If both endpoints have the INSIDE code, the entire line segment is accepted as it lies within the clipping window.
    - *Both Points Outside:* If both endpoints share an outside region (indicated by a bitwise AND of their codes), the line segment is rejected as it lies entirely outside the clipping window.
    - *One Point Inside, One Outside:* If one endpoint is inside and the other is outside, the algorithm determines where the line segment intersects with the clipping window. The intersection point is computed based on the outside endpoint's region code:
      - If the outside endpoint is above the clipping window (TOP), calculate the intersection with the top edge.
      - If below (BOTTOM), calculate with the bottom edge.
      - If to the right (RIGHT), calculate with the right edge.
      - If to the left (LEFT), calculate with the left edge.
    - The outside endpoint is replaced by the intersection point, and the region code for that endpoint is updated. This process repeats until both endpoints are either inside or the segment is rejected.
- *Drawing the Lines:*
  - After processing, the original line segment is drawn in one color (green), and if the segment was clipped, the visible portion is drawn in a different color (red).
- *Output:*
  - The final output consists of the clipping rectangle and the visual representation of the original and clipped line segments, providing a clear view of the clipping operation.

#### Code:

```
#include <GL/glut.h>
#include <iostream>
using namespace std;
// Define the clipping window (20, 20, 80, 80)
double x_min = 20, y_min = 20, x_max = 80, y_max = 80;
// Define region codes
```

```

const int INSIDE = 0; // 0000
const int LEFT = 1; // 0001
const int RIGHT = 2; // 0010
const int BOTTOM = 4; // 0100
const int TOP = 8; // 1000

// Global variables to store user
input for the line coordinates
double x_0, y_0, x_1, y_1;

// Function to compute the region
code for a point (x, y)
int computeCode(double x, double y){
    int code = INSIDE;

    if (x < x_min){
        code |= LEFT;    }

    else if (x > x_max){
        code |= RIGHT;    }

    if (y < y_min){
        code |= BOTTOM;    }

    else if (y > y_max){
        code |= TOP;    }

    return code;}

// Cohen-Sutherland clipping
algorithm

void cohenSutherlandClip(double x_0,
double y_0, double x_1, double y_1){

    int code_0 = computeCode(x_0,
y_0);

    int code_1 = computeCode(x_1,
y_1);

    bool accept = false;

    while (true) {

        if (!(code_0 | code_1)) {
// Both points inside

            accept = true;

            break;}

```

```

else if (code_0 & code_1) { // Both
points share an outside region,
trivial reject

        break;    } else {

        // Some segment of the
line is within the clip window

        double x, y;

        int codeOut = code_0 ?
code_0 : code_1;

        if (codeOut & TOP) {

            x = x_0 + (x_1 -
x_0) * (y_max - y_0) / (y_1 - y_0);

            y = y_max;
        } else if (codeOut & BOTTOM) {

            x = x_0 + (x_1 -
x_0) * (y_min - y_0) / (y_1 - y_0);

            y = y_min;
        } else if (codeOut & RIGHT) {

            y = y_0 + (y_1 -
y_0) * (x_max - x_0) / (x_1 - x_0);

            x = x_max;

        } else if (codeOut &
LEFT) {

            y = y_0 + (y_1 -
y_0) * (x_min - x_0) / (x_1 - x_0);

            x = x_min;

        }

        // Replace point outside
the window

        if (codeOut == code_0) {

            x_0 = x;

            y_0 = y;

            code_0 =
computeCode(x_0, y_0);

        } else {

            x_1 = x;

            y_1 = y;

            code_1 =

```

```

        code_1 =
computeCode(x_1, y_1); }}}

    if (accept) {

        // Draw the original line
        outside the clipping window
        glColor3f(0.0, 1.0, 0.0); // Green

        glBegin(GL_LINES);

        glPointSize(3.0);

        glVertex2d(x_0, y_0);
        glVertex2d(x_1, y_1);

        glEnd();

        // Draw the clipped line
        inside the clipping window
        glColor3f(0.0, 1.0, 0.0); // Red

        glBegin(GL_LINES);

        glVertex2d(x_0, y_0);
        glVertex2d(x_1, y_1);

        glEnd();}}

void display() {

    glClear(GL_COLOR_BUFFER_BIT);

    glMatrixMode(GL_PROJECTION);

    glLoadIdentity();

    // Set up the coordinate system
    gluOrtho2D(0, 100, 0, 100);

    glMatrixMode(GL_MODELVIEW);

    // Draw the clipping rectangle
    (20, 20, 80, 80)

    glColor3f(1.0, 1.0, 0.0); //
yellow

    glBegin(GL_LINE_LOOP);

    glVertex2d(x_min, y_min);

    glVertex2d(x_max, y_min);

    glVertex2d(x_max, y_max);

    glVertex2d(x_min, y_max);

    glEnd();

```

```

    // Draw the initial line before
    clipping

    glColor3f(1.0, 0.0, 0.0); //
Red

    glBegin(GL_LINES);

    glVertex2d(x_0, y_0);

    glVertex2d(x_1, y_1);

    glEnd();

    // Perform clipping and draw the
    clipped part in red

    cohenSutherlandClip(x_0, y_0,
x_1, y_1);

    glFlush();

}

int main(int argc, char** argv) {

    cout << "Enter the coordinates
of the line (x_0, y_0, x_1, y_1): ";

    cin >> x_0 >> y_0 >> x_1 >> y_1;

    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_SINGLE
| GLUT_RGB);

    glutInitWindowSize(500, 500);

    glutCreateWindow("Cohen-
Sutherland Line Clipping
Algorithm");

    glClearColor(0.0, 0.0, 0.0,
1.0);

    glutDisplayFunc(display);

    glutMainLoop();

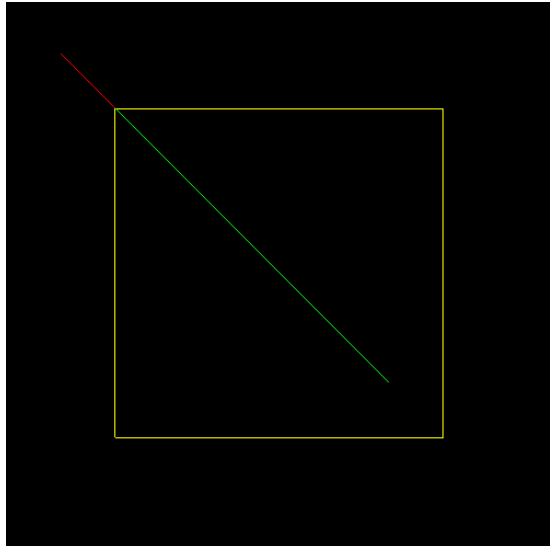
    return 0;

}

```

**Output:**

Enter the coordinates of the line (x\_0, y\_0, x\_1, y\_1): 10 10 70 70

**Discussion:**

During the implementation of the Cohen-Sutherland line clipping algorithm, I faced some issues with Line Intersection Calculations. When calculating the intersection points for lines outside the clipping window, I initially failed to correctly implement the formulas. Specifically, for vertical or horizontal lines, the slope calculation resulted in division by zero or incorrect intersection points because I didn't account for cases where the line segment was parallel to one of the boundaries. I corrected this by separately handling cases where the line is either completely vertical or horizontal, ensuring the intersection calculations were accurate. And, I mixed Up of coordinates in Drawing Functions. While rendering the line segments, I mistakenly swapped some of the coordinates in the glVertex2d() function, which caused the lines to appear inverted or shifted in unexpected ways. This was particularly problematic when drawing the original unclipped line and the final clipped portion. Initially, the code either rejected or accepted lines early. This happened because I did not properly handle the bitwise operations on the region codes. I learned that to correctly implement the accept/reject logic, the bitwise AND operation (&) should be used to test whether both points lie outside the same boundary, while the bitwise OR operation (|) should be used to determine if at least one point lies outside. Proper use of these operators helped in correctly identifying the cases for trivial acceptance or rejection.

**Experiment no. 06**

**Title:** Sutherland-Hodgman Polygon Clipping Algorithm

**Objective:**

The Sutherland-Hodgman algorithm is a fundamental technique in computer graphics for clipping polygons against a rectangular clipping window. This implementation allows us to understand the principles of polygon clipping, specifically how the algorithm handles vertices inside and outside the clipping window. By coding this algorithm, we can evaluate its performance and visualize the differences between the original polygon and the clipped polygon.

### Algorithm:

The Sutherland-Hodgman algorithm is used to clip a polygon by a rectangular clipping window. It clips the polygon against each edge of the window one by one. The algorithm takes each edge of the polygon and clips it according to whether the vertices are inside or outside the clipping boundary.

Here's the step-by-step algorithm:

- *Start with the polygon:* The polygon is defined by its vertices, and it can have any number of sides. The clipping window is defined as a rectangle with its four edges (left, right, top, bottom).
- *Clip against each edge:* The polygon is clipped against the four edges of the rectangle (left, right, bottom, top) one by one. After clipping against each edge, we update the polygon's vertices to reflect the changes.
  - We only clip against one boundary at a time.
- *Check each vertex:*
  - For each pair of consecutive vertices of the polygon, check whether they are inside or outside the clipping edge.
  - There are four cases to consider:
    - *Both vertices are inside:* The next vertex is added to the output list.
    - *First vertex is inside, second vertex is outside:* Add the intersection point between the edge and the clipping boundary to the output list.
    - *First vertex is outside, second vertex is inside:* Add the intersection point and the second vertex to the output list.
    - *Both vertices are outside:* No points are added to the output list.
- *Intersection calculation:*
  - If the line between two vertices intersects the clipping edge, compute the exact point of intersection.
  - *For vertical clipping (left or right):*
    - If clipping against a vertical line (like the left or right edge), the x-coordinate is fixed at the clipping boundary (e.g.,  $x = x_{\min}$  or  $x = x_{\max}$ ).
    - The y-coordinate of the intersection is calculated by using the slope of the line between the two vertices:
 
$$y_{\text{intersection}} = y_1 + (y_2 - y_1) \times \frac{(x_{\text{clip}} - x_1)}{(x_2 - x_1)}$$
  - *For horizontal clipping (top or bottom):*
    - If clipping against a horizontal line (like the top or bottom edge), the y-coordinate is fixed at the clipping boundary (e.g.,  $y = y_{\min}$  or  $y = y_{\max}$ ).
    - The x-coordinate of the intersection is similarly calculated using:
 
$$x_{\text{intersection}} = x_1 + (x_2 - x_1) \times \frac{(y_{\text{clip}} - y_1)}{(y_2 - y_1)}$$
- *Repeat for all edges:*
  - The polygon is clipped against each of the four clipping boundaries (left, right, bottom, top). After clipping against one edge, the new clipped polygon is used for the next edge.
- *Final Result:* After clipping the polygon against all four edges, the final result will be the polygon (or part of it) that lies inside the clipping window.

**Code:**

```

#include <GL/glut.h>
#include <iostream>
using namespace std;

// Define the clipping window (20,
20, 80, 80)

double x_min = 20, y_min = 20, x_max
= 80, y_max = 80;

// Define region codes
const int INSIDE = 0; // 0000
const int LEFT = 1; // 0001
const int RIGHT = 2; // 0010
const int BOTTOM = 4; // 0100
const int TOP = 8; // 1000

// Global variables to store user
input for the line coordinates
double x_0, y_0, x_1, y_1;

// Function to compute the region
code for a point (x, y)
int computeCode(double x, double y) {
    int code = INSIDE;

    if (x < x_min){
        code |= LEFT;    }

    else if (x > x_max){
        code |= RIGHT;    }

    if (y < y_min){
        code |= BOTTOM;    }

    else if (y > y_max){
        code |= TOP;    }

    return code;}

// Cohen-Sutherland clipping
algorithm

void cohenSutherlandClip(double x_0,
double y_0, double x_1, double y_1) {
    int code_0 = computeCode(x_0,

```

```

    int code_0 = computeCode(x_0, y_0);
    int code_1 = computeCode(x_1, y_1);

    bool accept = false;

    while (true) {
        if (!(code_0 | code_1)) { //
Both points inside
            accept = true;
            break;
        } else if (code_0 & code_1) {
// Both points share an outside
region, trivial reject
            break;
        } else {
            // Some segment of the
line is within the clip window

            double x, y;

            int codeOut = code_0 ?
code_0 : code_1;

            if (codeOut & TOP) {
                x = x_0 + (x_1 - x_0)
* (y_max - y_0) / (y_1 - y_0);
                y = y_max;
            } else if (codeOut & BOTTOM) {
                x = x_0 + (x_1 - x_0) *
(y_min - y_0) / (y_1 - y_0);
                y = y_min;
            } else if (codeOut & RIGHT) {
                y = y_0 + (y_1 - y_0)
* (x_max - x_0) / (x_1 - x_0);
                x = x_max;
            } else if (codeOut & LEFT) {
                y = y_0 + (y_1 - y_0)
* (x_min - x_0) / (x_1 - x_0);
                x = x_min;
            }
        }
    }
}

```

```

// Replace point outside the window
    if (codeOut == code_0) {
        x_0 = x;
        y_0 = y;
        code_0 =
computeCode(x_0, y_0);
    } else {
        x_1 = x;
        y_1 = y;
        code_1 =
computeCode(x_1, y_1); }}    }

    if (accept) {
        // Draw the original line
        outside the clipping window
        glColor3f(0.0, 1.0, 0.0); // Green
        glBegin(GL_LINES);
        glPointSize(3.0);
        glVertex2d(x_0, y_0);
        glVertex2d(x_1, y_1);
        glEnd();

        // Draw the clipped line
        inside the clipping window
        glColor3f(0.0, 1.0, 0.0); // Red
        glBegin(GL_LINES);
        glVertex2d(x_0, y_0);
        glVertex2d(x_1, y_1);
        glEnd();}}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    // Set up the coordinate system
    gluOrtho2D(0, 100, 0, 100);

```

```

    glMatrixMode(GL_MODELVIEW);
    // Draw the clipping rectangle
    (20, 20, 80, 80)
    glColor3f(1.0, 1.0, 0.0); //
yellow
    glBegin(GL_LINE_LOOP);
    glVertex2d(x_min, y_min);
    glVertex2d(x_max, y_min);
    glVertex2d(x_max, y_max);
    glVertex2d(x_min, y_max);
    glEnd();

    // Draw the initial line before
    clipping
    glColor3f(1.0, 0.0, 0.0); // Red
    glBegin(GL_LINES);
    glVertex2d(x_0, y_0);
    glVertex2d(x_1, y_1);
    glEnd();

    // Perform clipping and draw the
    clipped part in red
    cohenSutherlandClip(x_0, y_0,
x_1, y_1);
    glFlush();
}

int main(int argc, char** argv) {
    cout << "Enter the coordinates of
the line (x_0, y_0, x_1, y_1): ";
    cin >> x_0 >> y_0 >> x_1 >> y_1;
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE |
GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Cohen-
Sutherland Line Clipping Algorithm");

```

```

glClearColor(0.0, 0.0, 0.0, 1.0);

glutDisplayFunc(display);

glutMainLoop();

return 0;}

// Function to compute the region
code for a point (x, y)
int computeCode(double x, double y) {
    int code = INSIDE;

    if (x < x_min){
        code |= LEFT;    }
    else if (x > x_max){
        code |= RIGHT;    }

    if (y < y_min){
        code |= BOTTOM;    }
    else if (y > y_max){
        code |= TOP;    }

    return code;}

// Cohen-Sutherland clipping
algorithm
void cohenSutherlandClip(double x_0,
double y_0, double x_1, double y_1) {
    int code_0 = computeCode(x_0,
y_0);

    int code_1 = computeCode(x_1,
y_1);

    bool accept = false;

    while (true) {
        if (!(code_0 | code_1)) { //
Both points inside
            accept = true;

            break;

        } else if (code_0 & code_1) {
// Both points share an outside
region, trivial reject

            break;

```

```

        } else {
            // Some segment of the
line is within the clip window

            double x, y;

            int codeOut = code_0 ?
code_0 : code_1;

            if (codeOut & TOP) {
                x = x_0 + (x_1 - x_0)
* (y_max - y_0) / (y_1 - y_0);

                y = y_max;

            } else if (codeOut &
BOTTOM) {
                x = x_0 + (x_1 - x_0)
* (y_min - y_0) / (y_1 - y_0);

                y = y_min;

            } else if (codeOut &
RIGHT) {
                y = y_0 + (y_1 - y_0)
* (x_max - x_0) / (x_1 - x_0);

                x = x_max;

            } else if (codeOut &
LEFT) {
                y = y_0 + (y_1 - y_0)
* (x_min - x_0) / (x_1 - x_0);

                x = x_min;
            }

            // Replace point outside
the window

            if (codeOut == code_0) {
                x_0 = x;

                y_0 = y;

                code_0 =
computeCode(x_0, y_0);

            } else {
                x_1 = x;

                y_1 = y;

                code_1 = computeCode(x_1, y_1);} }}

```

```

    if (accept) {
        // Draw the original line
        outside the clipping window

        glColor3f(0.0, 1.0, 0.0); //
Green
        glBegin(GL_LINES);
        glPointSize(3.0);
        glVertex2d(x_0, y_0);
        glVertex2d(x_1, y_1);
        glEnd();

        // Draw the clipped line
        inside the clipping window

        glColor3f(0.0, 1.0, 0.0); //
Red
        glBegin(GL_LINES);
        glVertex2d(x_0, y_0);
        glVertex2d(x_1, y_1);
        glEnd();    }}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Set up the coordinate system
    gluOrtho2D(0, 100, 0, 100);
    glMatrixMode(GL_MODELVIEW);

    // Draw the clipping rectangle
    (20, 20, 80, 80)

    glColor3f(1.0, 1.0, 0.0); //
yellow
    glBegin(GL_LINE_LOOP);
    glVertex2d(x_min, y_min);
    glVertex2d(x_max, y_min);
    glVertex2d(x_max, y_max);
    glVertex2d(x_min, y_max);

```

```

    glEnd();

    // Draw the initial line before
    clipping

    glColor3f(1.0, 0.0, 0.0); // Red
    glBegin(GL_LINES);
    glVertex2d(x_0, y_0);
    glVertex2d(x_1, y_1);
    glEnd();

    // Perform clipping and draw the
    clipped part in red

    cohenSutherlandClip(x_0, y_0,
x_1, y_1);
    glFlush();}

int main(int argc, char** argv) {
    cout << "Enter the coordinates of
the line (x_0, y_0, x_1, y_1): ";
    cin >> x_0 >> y_0 >> x_1 >> y_1;
    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_SINGLE |
GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Cohen-
Sutherland Line Clipping Algorithm");
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```

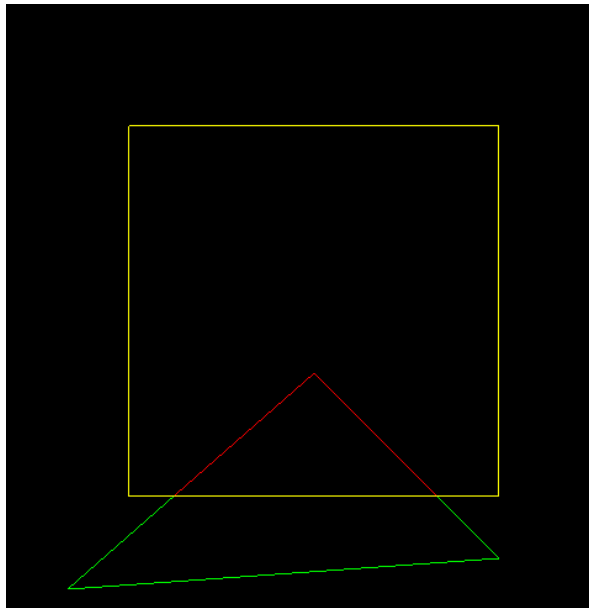
**Output:**

Enter the number of sides of the polygon: 3

Enter coordinates for vertex 1 (x, y): 10 5

Enter coordinates for vertex 2 (x, y): 50 40

Enter coordinates for vertex 3 (x, y): 80 10

**Discussion:**

During the implementation of the Sutherland-Hodgman polygon clipping algorithm, I faced a few issues and learned important lessons. One of the main challenges was correctly calculating the intersection points when a polygon edge crossed the boundary of the clipping rectangle. Initially, I misunderstood how to compute the intersection, especially when the polygon was almost parallel to one of the clipping edges. The correct approach involved using the equation of a line to find the precise intersection point with the boundary. Managing Multiple Clipping Passes: Since the algorithm clips the polygon against all four boundaries (left, right, bottom, and top) sequentially, I initially struggled with the fact that each clipping step modifies the polygon and feeds the result into the next step. I didn't initially account for how the vertices needed to be updated after clipping against one boundary before moving to the next. This led to errors where parts of the polygon were missing. After understanding this process, I implemented the steps correctly and got the right output. I initially tested the algorithm on convex polygons, which are simpler to clip since they do not have any inward "dents". However, when I tried clipping concave polygons (which have inward parts), I noticed that the clipped shape was not correct. I learned that the algorithm works for both convex and concave polygons, but special attention needs to be given to ensure that the order of vertices and intersection points is handled correctly, as concave shapes introduce more complexity. To visually confirm if the clipping was working properly, I drew both the original polygon and the clipped polygon using different colors. At first, it was difficult to see the clipping effect because I mistakenly drew both polygons on top of each other using the same color. By fixing this and assigning different colors to the original and clipped polygons, it became much easier to verify that the clipping was occurring as expected.